Task 7: Type Conversion and Precision

Student name	Sk.Thohith	Register number	1912260555
GitHub repository	https://github.com/[username]/CSA02-[RegisterNumber]-Assignment1/tree/main/Task07	Compile command	gcc -Wall -Wextra -std=c11 -O2 main.c -o task
Compiler	Online C compiler (screenshot); own checks GCC 13.3.0	Date	26 October 2026

1.1 Problem analysis

The program demonstrates nine situations where C changes or loses information when types are mixed: integer vs floating division, implicit conversion, explicit casting, the 0.1 + 0.2 representation error, rounding at display time, narrowing a `long` to `int`, and accumulation drift of `float` vs `double` over 10000 additions. The aim is to explain *why* each output appears and to choose safe types for money and measurements, which links to Tasks 10-12 (billing in integer paise).

1.2 Data types, constants and preprocessor concepts

Item	Type / form	Role in the program
a, b	int = 5, 2	Operands for integer division; a / b = 2 (fraction discarded)
x, y	float = 5.0f, 2.0f	Floating division gives 2.50
amount	double = 123.456	%.3f and %.2f print the same stored value differently (rounding only at display)
large_value	long = 3000000000L	Needs 64-bit long (8 bytes here); exceeds 32-bit int
INT_MAX	Macro from <limits.h>, 2147483647	Boundary for the safe-cast test (line 42)
float_total, double_total	float, double accumulators	Compare precision over 10000 additions of 0.1
#include <stdio.h>, <limits.h>	Preprocessor directives	Textually include printf and INT_MAX declarations before compilation
%d, %.2f, %.17f, %ld, %.6f	printf specifiers	Must match the argument type; a float argument is promoted to double

1.3 Source code (main.c, lines 1-66 as shown; ends of lines clipped by the editor are marked ...)

```
#include <stdio.h>
#include <limits.h>
int main(void)
{
    int a = 5, b = 2;
    float x = 5.0f;  float y = 2.0f;
    double amount = 123.456;
    long large_value = 3000000000L;

    printf("TASK 7: TYPE CONVERSION AND PRECISION\n");
    printf("-------------------------------------\n");

    /* 1. Integer division */
    printf("Integer division (5 / 2) = %d\n", a / b);
    /* 2. Floating-point division */
    printf("Float division (5.0 / 2.0) = %.2f\n", x ...);
    /* 3. Implicit conversion */
    printf("Implicit conversion (5 / 2.0) = %.2f", a / 2.0);
    /* 4. Explicit type casting */
    printf("Explicit casting (5 / 2) = %.2f\n", (double)a / b);
    /* 5. Integer division surprise */
    printf("Integer division (7 / 3) = %d\n", 7 / 3);
    /* 6. Floating-point precision */
```

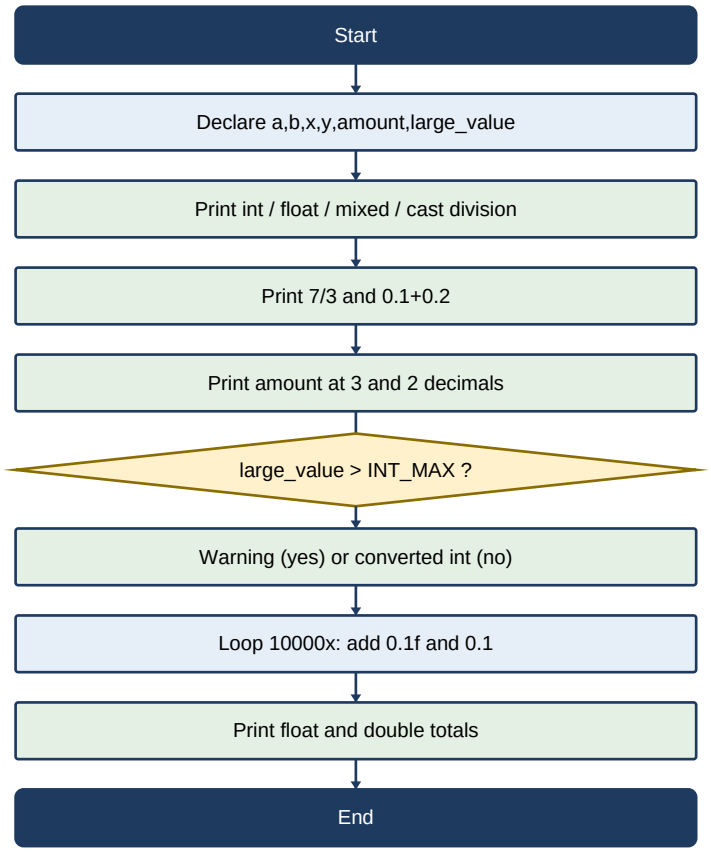
```
    printf("0.1 + 0.2 = %.17f\n", 0.1 + 0.2);
    /* 7. Rounding to two decimal places */
    printf("Amount before display rounding = %.3f\n" ...);
    printf("Amount displayed to 2 decimals = %.2f\n" ...);
    /* 8. Casting a large long to int */
    printf("Large value stored in long = %ld\n", lar...);
    if (large_value > INT_MAX)
    {   printf("Warning: value exceeds INT_MAX; conv...);   }
    else
    {   int converted = (int)large_value;
        printf("Converted integer = %d\n", converted);   }
    /* 9. Float versus double accumulation */
    float float_total = 0.0f;
    double double_total = 0.0;
    for (int i = 0; i < 10000; i++)
    {   float_total += 0.1f;
        double_total += 0.1;   }
    printf("Float total after 10000 additions = %.6f ...);
    printf("Double total after 10000 additions = %.1 ...);
    return 0;
}
```

The clipped argument lists could not be read from the screenshot; copy the complete lines from your own main.c if you want the full listing in the submission.

1.4 Pseudocode

```
a, b <- 5, 2 ; x, y <- 5.0, 2.0 ; amount <- 123.456 ; large <- 3000000000
PRINT a DIV b                       // integer division -> 2
PRINT x / y                         // real division    -> 2.50
PRINT a / 2.0                       // a converted to real (implicit)
PRINT (double)a / b                 // a converted to real (explicit cast)
PRINT 7 DIV 3 ; PRINT 0.1 + 0.2 with 17 decimals
PRINT amount with 3 decimals, then with 2 decimals (rounded for display only)
IF large > INT_MAX THEN PRINT warning ELSE PRINT (int)large
float_total <- 0 ; double_total <- 0
REPEAT 10000 times: float_total += 0.1 ; double_total += 0.1
PRINT both totals
```

UML activity diagram - Task 7



1.5 Output evidence

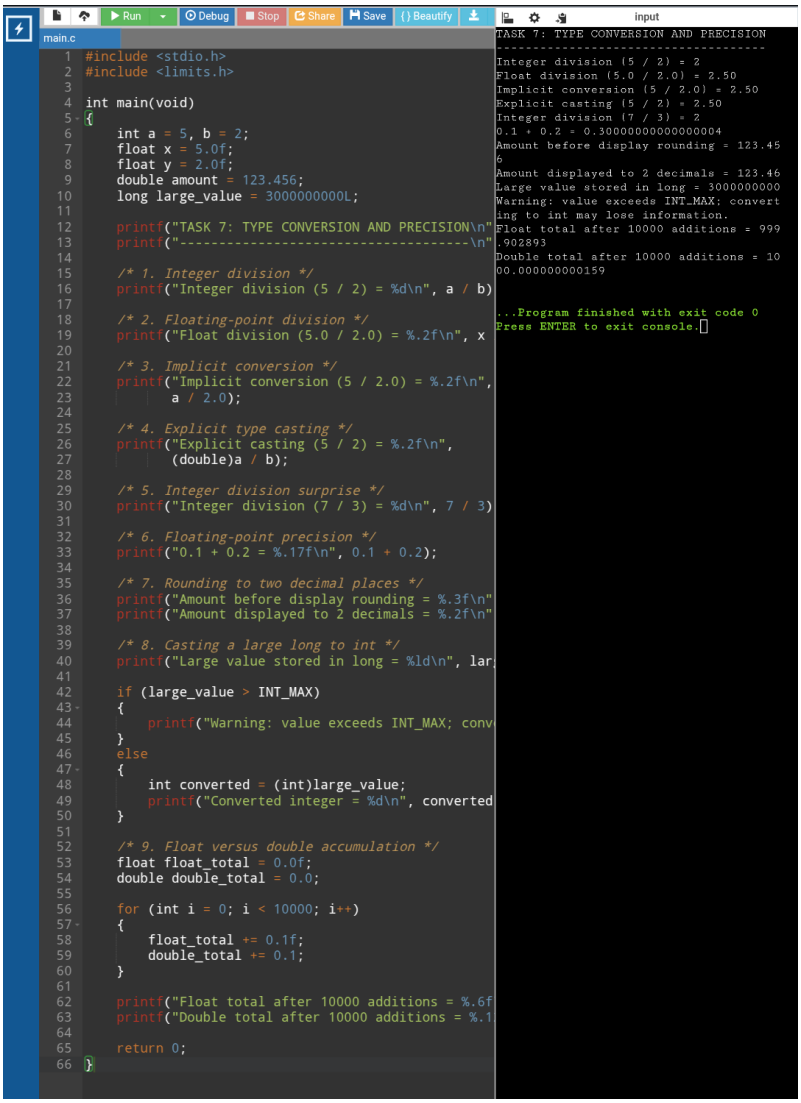


Figure 1 - Task 7 source (left) and console output (right).

1.6 Explanation of each output and test table

#	Case	Expected (reasoning)	Actual	Result
1	5 / 2 (int, int)	Both operands int, so the quotient is truncated: 2	2	Pass
2	5.0f / 2.0f	Real division: 2.50	2.50	Pass
3	5 / 2.0	int 5 is converted to double: 2.50	2.50	Pass
4	(double)5 / 2	Cast applies to a before the division: 2.50	2.50	Pass
5	7 / 3	Truncated toward zero: 2	2	Pass
6	0.1 + 0.2	Neither value is exact in binary: not exactly 0.3	0.30000000000000004	Pass
7	123.456 at %.3f and %.2f	Stored value unchanged; printing rounds: 123.456 and 123.46	123.456 / 123.46	Pass
8	3000000000L > INT_MAX	True (3,000,000,000 > 2,147,483,647): warning branch	Warning printed	Pass
9	10000 x 0.1	Exact answer 1000. float drifts, double is much closer	float 999.902893; double 1000.000000000159	Pass

Additional boundary checks (my own run, GCC 13.3.0 on a 64-bit Linux sandbox; repeat on your machine).

Case	Result	Meaning
large_value = INT_MAX (2147483647)	Takes the else branch: Converted integer = 2147483647	Largest value that converts safely
large_value = INT_MAX + 1	Warning branch	First value that does not fit
(int)3000000000L, if the guard were absent	-1294967296	Wraps around (implementation-defined); the guard in the program prevents this
-7 / 2 and -7 % 2	-3 and -1	Division truncates toward zero in C99 and later
-123.456 and 2.675 at %.2f	-123.46 and 2.67	2.675 is stored slightly below 2.675, so it rounds down

Float accumulation error. Exact total 1000. Float printed 999.902893, an error of 0.097107 (0.0097%); I reproduced this exact value on GCC 13.3.0. Double printed 1000.000000000159, an error of about 1.59e-10, roughly 600 million times smaller. The reason: 0.1f is stored as 0.100000001490116..., and a float has only about 7 significant digits, so once the running total is in the hundreds each addition is rounded to a coarse step.

Portability note. `long` is 8 bytes on 64-bit Linux (as here) but 4 bytes on 64-bit Windows compilers, where 3000000000L would not fit. Use `long long` or `int64_t` when 64 bits are required.

1.7 Alternative concept: scaled integers and compensated summation instead of float/double accumulation

Two alternatives remove the drift in test 9. **(a) Scaled integer:** count tenths in a `long long` and divide only for display. **(b) Kahan compensated summation:** keep a running correction term so that lost low-order bits are fed back into the next addition.

```
long long tenths = 0;                       /* (a) exact */
for (int i = 0; i < 10000; i++) tenths += 1;
printf("%lld.%lld\n", tenths / 10, tenths % 10);   /* 1000.0 */

float sum = 0.0f, c = 0.0f;                 /* (b) Kahan */
for (int i = 0; i < 10000; i++) {
    float y = 0.1f - c;  float t = sum + y;
    c = (t - sum) - y;   sum = t;
}
```

Results from my run: scaled integer printed exactly 1000.0; Kahan with float printed 1000.000000 (naive float: 999.902893). Kahan can be defeated by aggressive floating-point optimisation such as `-ffast-math`, which may remove the compensation.

	Naive float	Naive double	Scaled integer	Kahan (float)
Time complexity	O(n)	O(n)	O(n)	O(n), about 4 flops per element
Space complexity	O(1)	O(1)	O(1)	O(1) (one extra variable)
Error after 10000 additions of 0.1	0.097107	1.59e-10	0 (exact)	about 0 at 6 decimals
Storage per value	4 bytes	8 bytes	8 bytes (long long)	4 bytes (+4 for correction)

Measurement (my loop of 10^8 additions, GCC 13.3.0, -O0, single run on a cloud sandbox): float 0.315 s, double 0.300 s, integer 0.213 s. All three scale linearly, confirming O(n); the integer loop is fastest. The run also shows a severe float failure at scale: after 10^8 additions of 0.1f the float sum stopped at 2097152.0 (2^{21}) because adding 0.1 no longer changes the value, while the double sum was 9999999.9811 against an exact 10,000,000.

1.8 Design conclusions

Situation	Recommended practice	Evidence from this task
Division of integers where a fraction matters	Cast one operand first: (double)a / b	Tests 1, 3, 4

Situation	Recommended practice	Evidence from this task
Money and counts	Integer paise or scaled integers; format at output	Tests 6, 9; ties to Tasks 10-12
Narrowing conversions	Guard with a limits.h comparison before casting	Test 8; (int)3000000000L = -1294967296
Long accumulations of real values	double, or Kahan, or scaled integers; avoid float	Test 9 and the 10^8 loop
Display rounding	Round only when printing; keep full precision internally	Test 7